

ENGR 102 Summer Bridge

Day 15 Instructor Key

Strings as Sequences, Collection, and Slicing

20 points • approximately 20-25 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as [and](#), [or](#), and [not](#). Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace sequence state and direct slicing.

4 points

```
text = "bridge"
pieces = []

for index in range(0, len(text), 2):
    pieces.append(text[index])

middle = text[1:5]
result = pieces + [middle]
print(pieces)
print(middle)
print(result)
```

Give the index ledger, slice boundaries, and every exact output line.

Answer and explanation

Indexes 0, 2, and 4 store b, i, and g, so pieces is ['b', 'i', 'g']. text[1:5] includes indexes 1 through 4 and is ridg. result is ['b', 'i', 'g', 'ridg']; these are the three output lines.

Question 2. Repair list/string concatenation.

3 points

```
items = ["x"]
items = items + "y"
print(items)
```

Name the error and give both a concatenation repair and an append repair.

Answer and explanation

The expression raises TypeError because list concatenation requires another list. Use items = items + ['y'] or items.append('y'); either produces ['x', 'y'].

Question 3. Program construction**13 points**

Write one complete Python program that satisfies every requirement.

- Read word as text and print invalid if its length is less than 3.
- Otherwise append characters at odd indexes to odd_characters.
- Count lowercase t characters.
- Build frame from the first one-character slice plus the last one-character slice.
- Print Odd, Lowercase t, and Frame on separate labeled lines. Include an iteration test with expected output.

Answer and explanation

```
word = input()

if len(word) < 3:
    print("invalid")
else:
    odd_characters = []
    t_count = 0
    for index in range(len(word)):
        character = word[index]
        if index % 2 == 1:
            odd_characters.append(character)
        if character == "t":
            t_count = t_count + 1
    frame = word[:1] + word[len(word) - 1:]
    print(f"Odd: {odd_characters}")
    print(f"Lowercase t: {t_count}")
    print(f"Frame: {frame}")
```

The length guard protects sequence work. An index loop is required because collection depends on index parity; t_count depends on the stored character value.

For iteration, odd indexes 1, 3, 5, and 7 store t, r, t, and o. The lowercase t count is 2, and first/last slices concatenate to in.

Scoring guidance

2 validation, 3 traversal/state, 3 odd-index collection, 2 t count, 2 slice/concatenation/output, 1 complete test.